

PhysiGym: bridging the gap between the Gymnasium reinforcement learning application interface and the PhysiCell agent-based model software

Alexandre Bertin ^{1,2,5,*}, Elmar Bucher ^{3,†}, Owen Griere^{2,5}, Marcelo Hurtado ^{2,5}, Heber Rocha ³, Randy Heiland ³, Aneequa Sundus ³, Paul Macklin ^{3,‡}, Vincent François-Lavet ^{4,‡}, Emmanuel Rachelson ^{1,‡}, and Vera Pancaldi ^{2,5,*}

¹CRCT, Université de Toulouse, Inserm, CNRS, Université Toulouse III-Paul Sabatier, Centre de Recherches en Cancérologie de Toulouse, France

²ISAE-Supaéro, Toulouse, France

³Department of Intelligent Systems Engineering, Indiana University, Bloomington, IN, USA

⁴Department of Computer Science, VU Amsterdam, The Netherlands

⁵Equipe Labellisée LIGUE Contre le Cancer, France

*Corresponding authors: alexandre.bertin@isae-superaero.fr, vera.pancaldi@inserm.fr

[†]These authors contributed equally to this work.

[‡]These authors contributed equally to this work.

September 16, 2025

Abstract

This paper presents PhysiGym, a framework that integrates agent-based biological simulation within standardized reinforcement learning environments. By integrating the agent-based modeling framework PhysiCell with the Gymnasium API, we provide a flexible tool for exploring reinforcement learning strategies to control in-silico biological processes. We demonstrate PhysiGym’s potential with a case study where a deep reinforcement learning algorithm guides a tumor microenvironment model toward an anti-tumoral state, ultimately achieving tumour elimination. Our results highlight PhysiGym’s flexibility for AI-driven biological control and optimization of dynamic treatment regimes.

1 Introduction

Biological systems are complex, dynamic, and often difficult to study in real life. In silico modeling aims to mimic these systems, providing a controlled environment for exploring specific aspects of biological processes and testing hypotheses. In particular, agent-based models (ABMs) provide a powerful framework for simulating biological systems where agents (e.g. cells) interact dynamically based on predefined rules. These models are used in oncology and immunology to study emerging behaviors in complex biological systems, with the potential to propose novel therapeutic strategies [11, 12, 25].

Ordinary differential equations (ODEs) and agent-based models (ABMs) are widely used to simulate the tumor microenvironment (TME). ODEs provide compact system-level descriptions, while ABMs capture spatial and cellular heterogeneity at higher computational cost. For dynamic treatment regimes (DTRs) [8], however, simulation alone is insufficient: designing adaptive, sequential interventions requires a control framework. Reinforcement learning (RL), where an RL agent learns policies through interaction with an environment, offers such a framework [2]. By modeling treatment regimes as a sequence of decisions, RL enables adaptive control of biological simulations [4]. For efficient experimentation and policy learning, RL further requires a standardized interface. To address this, we introduce PhysiGym, an interface that connects PhysiCell [7], an ABM framework for multicellular simulations of tissues or cell cultures (tumor microenvironment, other disease tissues, or bacteria colonies), with Gymnasium [28], a widely adopted RL application programming interface (API). PhysiCell enables flexible multiscale modeling of biological systems, coupled with a sophisticated underlying physics simulator (the BioFVM diffusion transport solver [6]), while Gymnasium provides a structured framework for training and evaluating RL policies.

The first implementation of RL on an ABM framework based on BioFVM was proposed by Zade et al. [33], which applied Q-learning [32] to optimize Temozolomide treatment regimes for patients with glioblastoma multiforme, a highly specific problem. More recently, Aif et al. [1] used PhysiCell as a complex (high-fidelity) simulation environment for tumor growth, combined with a simple (low-fidelity) RL training environment of coupled ordinary differential equations (ODE) to learn treatment strategies for solid tumors.

PhysiGym bridges PhysiCell and Gymnasium to provide a reproducible and scalable framework for integrating agent-based simulations of biological systems with RL strategies. In the following, we present PhysiGym’s motivation and design, demonstrating how it directly connects ABMs and RL to control complex biological systems. In particular, we focus on the tumor microenvironment, a biological system made up of cancer cells and surrounding immune and stromal cells that engage in complex interactions and cross-talks, which is proven to be crucial for an effective response to immunotherapy, an important therapeutic approach in an increasing number of types of cancer.

2 Methods

2.1 PhysiGym design and implementation

PhysiCell is an ABM framework written in C++ and implemented to model multicellular systems based on classical mechanics [7]. Cells are agents whose type determines the specific rule set that governs their behavior and interactions with the other agents and the environment. Substrates, like oxygen or cytokines, can be modeled with the integrated BioFVM diffusive transport solver [6]. In addition, intracellular models can be integrated into cell agents [14, 21, 26] to model specific behaviors based on each cell’s environment and gene regulatory networks. The extracellular matrix that surrounds cells in tissues can also be modeled as a mesh of fibers for a more realistic representation of the cells’ physical environment [19]. Together, these components enable the construction of ABMs that are spatially explicit in two or three dimensions, off-lattice, center-based, and multiscale in both space and time [16].

RL is used to control discrete-time dynamical systems, which are typically modeled as Markov Decision Processes (MDPs) [3, 22]. An MDP consists of four key elements:

1. S the state space, where $s \in S$ represents a state of the environment.
2. A the action space, where $a \in A$ represents an action applied to the environment (e.g. drug dose).
3. T the transition model, which defines how the environment evolves. The transition model can be either deterministic or stochastic. Formally, the next state noted s' is given by $s' \sim T(s, a)$ where s represents the current state and a the action taken.
4. R the reward function, which evaluates whether an action was beneficial. Formally, r denotes the reward obtained by taking action a in state s , i.e., $r = R(s, a)$.

The objective is to find the policy that maximizes the expected discounted cumulative reward:

$$\arg \max_{\pi \in \Pi} \mathbb{E} \left[\sum_{t=0}^{\tau} \gamma^t r_t \mid s_0 = s, \pi \right],$$

with s_0 the initial state, π the policy, γ the discount factor, and τ the terminal timestep. In the RL community, the Gymnasium RL API is widely used as a standard interface for MDPs, promoting code reuse and modularity in RL algorithms and models.

PhysiGym bridges the gap between the PhysiCell ABM framework and the Gymnasium RL API by extending the Python interpreter [29, 23] with PhysiCell. As a result, the ABM model is implemented in the same way as standard PhysiCell models, using PhysiCell Studio [10] and C++.

Any model implemented in PhysiCell can be converted into a PhysiGym model. The interface between PhysiCell and Gymnasium is based on the PhysiCell "user parameter", "custom data variable", and "custom data vectors" data constructs. Information, such as how much drugs to apply to the domain, can only be transferred over these variable types. The variables (for example, drug) can easily be specified in PhysiCell Studio under the **User Params** and **Cell Types / Custom Data** tab. As a minimum requirement, the user must define the parameter `dt_gym`, which specifies the time interval at which interactions with the environment occur, and the parameter `time`, a time tracker necessary to determine when to update the environment. In addition, additional parameters can be defined to control the simulation, such as drug administration, which can serve as actions. Similarly, other parameters can be designated to record specific quantities of interest, which can then be used to construct the observations.

According to the transition function, for each episode (a complete sequence of states, actions, and rewards from the start to the end of a trial), values from these data constructs can be modified without having to recompile the source code. There are three additional functions to define the state space (read information from

the simulation): `get_cell` for cell position, `get_microenv` for substrate concentration, and `get_graph` to retrieve cell neighborhood information.

To extend the Python interpreter with PhysiCell, the main simulation loop, written in C++, was split into three functions: `physicell_start`, `physicell_step`, and `physicell_stop`, all of which can be called from Python. The `physicell_start` function initializes the PhysiCell model, while `physicell_stop` terminates the simulation. The `physicell_step` function is responsible for advancing the environment by one time step based on the action taken by the reinforcement learning agent. For most models, only `physicell_step` needs to be modified, e.g. to apply drugs to the ABM. Several additional C++ functions are exposed in Python to facilitate data retrieval. These include functions to obtain custom variables, vectors, `physicell_get_cell`, which provides a matrix containing information about each cell, such as its type, coordinates, and death status, and `physicell_get_microenv`. All exposed functions are implemented in the C++ file `physicellmodule.cpp`.

By extending the necessary functions to obtain the desired control over the ABM and modifying the `physicell_step` function, users can compile the C++ code and integrate it with Python, particularly with the Gymnasium API. This allows users to define `physicell_model.py`, which implements a Gymnasium-compatible environment. Specifically, `physicell_model.py` defines a child class of Gymnasium’s environment class, inheriting from `CorePhysiCellEnv` which is specified in `physicell_core.py`. `CorePhysiCellEnv` is the main class that takes charge of the use of `physicell_start`, `physicell_stop` and `physicell_step` which are model independent. This class handles interactions with the Gymnasium framework. Users should modify `physicell_model.py` to specify the observation space, action spaces, and the reward, as well as any additional data they wish to store.

For more details, consult the reference manual <https://github.com/Dante-Berth/PhysiGym/blob/main/man/REFERENCE.md>.

For an in-depth understanding, tutorials are provided and can be found at the PhysiGym GitHub home page. Besides, pytest unit tests are included in the code base, and we use GitHub Actions for continuous integration. PhysiGym was extensively tested for compiling and running on Linux, Windows Subsystem for Linux, and macOS. Installation and troubleshooting how-tos, a modeling tutorial, an RL tutorial, and a reference manual can be found at [<https://github.com/Dante-Berth/PhysiGym/tree/main/man>].

2.2 Implementation of a tumor microenvironment toy model 1

We developed a simplified tumor microenvironment (TME) model 1 to illustrate the potential uses of PhysiGym. The model includes four cell types, four diffusible substrates, and a set of simple agent-based rules. At the initial state, the total number of `tumor` cells is 512. And 128 `cell_1` surrounds the tumor. Moreover the total number of `cell_1` and `cell_2` is fixed at 128.

1. Cell types

- **Tumor cells:** proliferate ($5 \times 10^{-5} \text{ min}^{-1}$), apoptosis ($1 \times 10^{-6} \text{ min}^{-1}$), immotile, uptake anti- and pro-tumoral factors. Necrosis, motility, and baseline secretion disabled.
- **Cell_1:** non-proliferative, mobile, uptakes pro-tumoral factor (10/min). Conditional rules allow secretion of anti-tumoral factor and transformation into `cell_2`.
- **Cell_2:** non-proliferative, mobile, uptakes `drug_1` (10/min), produces pro-tumoral. Conditional rules allow transformation into `cell_1` in presence of `drug_1`.

2. Substrates

1. **Debris:** produced by dead tumor cells, diffusion $1 \mu\text{m}^2/\text{min}$, no decay. Acts as a signal for debris-related rules.
2. **Drug_1:** decay 0.01/min, applied at boundaries, regulates `cell_2` $\rightarrow$ `cell_1` transformation.
3. **Anti-tumoral factor/ pro-inflammatory factor:** diffusion $1000 \mu\text{m}^2/\text{min}$, decay 0.05/min, secreted conditionally by `cell_1`, enhances tumor apoptosis.
4. **Pro-tumoral factor/ anti-inflammatory factor:** diffusion $1000 \mu\text{m}^2/\text{min}$, decay 0.1/min, is produced by `cell_2` and enhances tumor survival.

3. Cell Hypothesis Rules

In `tumor` cells: pressure decreases cycle entry from $5\text{e-}05$ towards 0 with a Hill response, with half-max 5 and Hill power 3. pressure increases necrosis from 0 towards 0.0028 with a Hill response, with half-max 10 and Hill power 3. anti-inflammatory factor decreases apoptosis from $1\text{e-}06$ towards 0 with a Hill response, with half-max 5 and Hill power 10. pro-inflammatory factor increases apoptosis from $1\text{e-}06$ towards 0.05 with a Hill response, with half-max 1 and Hill power 2. dead increases debris secretion from 0 towards 0.25 with a Hill response, with half-max 0.2 and Hill power 5. Rule applies to dead cells.

In **cell_1** cells: pressure increases transform to **cell_2** from 0 towards 1 with a Hill response, with half-max 11 and Hill power 4. anti-inflammatory factor decreases pro-inflammatory factor secretion from 10 towards 0 with a Hill response, with half-max 10 and Hill power 4.

In **cell_2** cells: **drug_1** increases transform to **cell_1** from 0 towards 1 with a Hill response, with half-max 5 and Hill power 4.

This toy model highlights how the RL framework can control tumor progression by modulating the whole microenvironment, rather than solely targeting cancer cell killing.

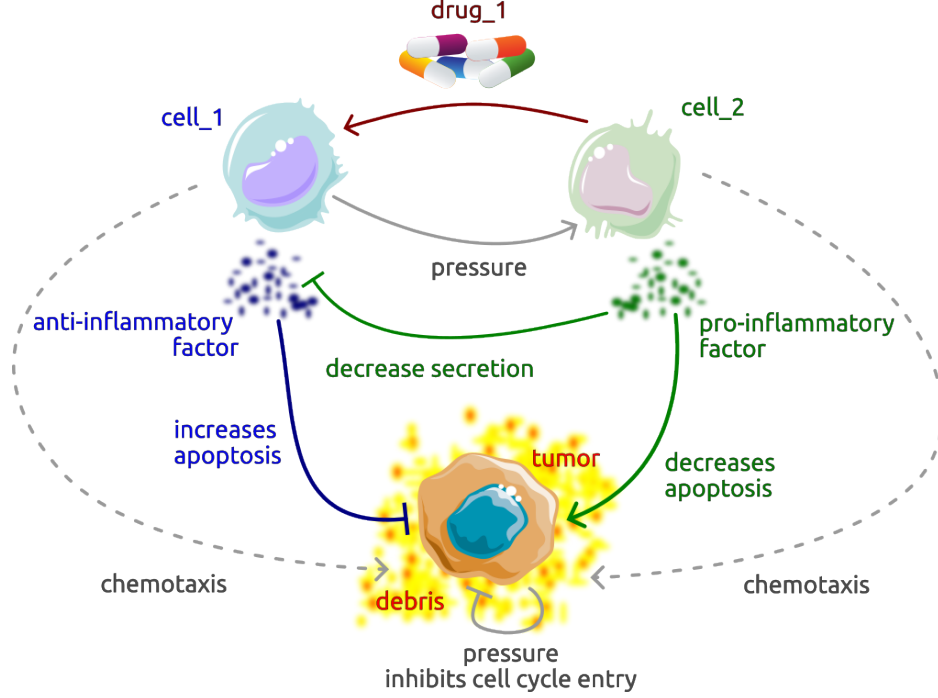


Figure 1: Toy model of the tumor microenvironment, representing interactions between cancer cells and two other distinct cell types with specific behaviors, together with exchanged cytokines and different processes. $\dashv$ represents an inhibition, while $\rightarrow$ represents an activation. The color of the arrows corresponds to the color of the cell producing the protein associated with the arrow. For instance, **cell_1** is in blue; thus, the arrow related to increasing apoptosis of the tumor is also in blue, as in **cell_1**.

2.3 Implementation of an RL approach to control the TME simulation

Our overall objective is to achieve complete tumor eradication while minimizing drug exposure and avoiding aggressive dosage escalation.

2.4 State space $\mathcal{S}$

In a tumor microenvironment, there are different types of cells, but also different molecules/chemical compounds viewed as substrates. Besides, it exists different types of data, data which gives an insight on the space and others data are spaceless such as a simple vector comporting some statistics about the tumor microenvironment. Overall, we propose to study in this model the importance of different state space on dynamic treatment regime in order to compare them on multiple metrics such as the cumulative return. The simulator provides multiple data, given that we can build different state spaces, we have considered scalars, images and graphs. We could ask, state space related to are superior to simple scalars related to space. We propose multiple different observation spaces, depending on whether we consider the spatial aspect of the simulation or not.

2.4.1 Scalars Observation Spaces

We denote by $\text{cell}_i(t)$ the number of cells of type i , with $i \in \{0, 1, 2\}$. Where $\text{cell}_1(t=0)$ refers to the number of tumor cells at the initial time, in our case it equals to 512. We use this term, to normalize any cell numbers. Let $\mathcal{T}$ denote the set of cell types and $|\mathcal{T}| = N_c$. Let $\mathcal{S}$ be the set of substrates with $|\mathcal{S}| = N_s$, in particular $\mathcal{S} = \{\text{debris}, \text{anti-inflammatory factor}, \text{pro-inflammatory factor}, \text{drug}_1\}$. Besides, we denote $\text{sub}_{j,t}$ as the amount of the substrate in the entire environment thus $\text{sub}_{j,t} \in (R^+)^{\text{width} \times \text{height}}$.

Scalar Cell Counts (scalars_cells). This observation space consists only of normalized cell counts for each cell type. For each cell type $i \in \mathcal{T}$, we compute

$$s_i^{\text{cells}}(t) = \frac{\text{cell}_i(t)}{c_{\text{init}}} - 1,$$

where c_{init} is the initial number of cancer cells. The resulting observation vector is

$$s_t^{\text{cells}} \in \mathbb{R}^{N_c}.$$

Scalar Substrate Values (scalars_substrates). This observation space captures only the maximum concentration of each substrate over the entire spatial domain. For each substrate $j \in \mathcal{S}$, we define

$$s_j^{\text{sub}}(t) = \max(\text{sub}_{j,t}),$$

where $\text{sub}_{j,t}$ denotes the spatial field of substrate j at time t . The observation is then

$$s_t^{\text{sub}} \in \mathbb{R}^{N_s}.$$

Scalar Cells and Substrates (scalars_cells_substrates). This representation concatenates the two previous scalar observations:

$$s_t^{\text{cells+sub}} = (s_t^{\text{cells}}, s_t^{\text{sub}}) \in \mathbb{R}^{N_c+N_s}.$$

2.4.2 Image-Based Observation Spaces

We consider the spatial distribution of cells and substrates via an image representation of the simulation state. A set of channels corresponds to a specific cell type, while the other set of channels refers to the substrates. Substrates are chemical products or extracellular proteins produced by cells or produced by the environment or chemical products added to the environment, such as drugs. To reduce memory requirements, we reduce the shape of the original image given by the `PhysiCell.Settings.xml` file by discretizing the continuous environment onto a uniform grid. We also compute $g_x = \lfloor \frac{\text{width}}{\text{gridsize}_x} \rfloor$ and $r_y = \lfloor \frac{\text{height}}{\text{gridsize}_y} \rfloor$. In our environment, $g_x = g_y$ because $\text{width} = \text{height} = 512$ and $\text{gridsize}_x = \text{gridsize}_y = \text{gridsize} = 64$. The size of the bins is calculated by mapping the continuous coordinates into discrete indices. Specifically:

$$x_{\text{bin}} = \left\lfloor \frac{(x - x_{\min})}{(x_{\max} - x_{\min})} \times (\text{gridsize}_x - 1) \right\rfloor,$$

$$y_{\text{bin}} = \left\lfloor \frac{(y - y_{\min})}{(y_{\max} - y_{\min})} \times (\text{gridsize}_y - 1) \right\rfloor.$$

This ensures that the continuous spatial domain is discretized into a grid of size $\text{gridsize} \times \text{gridsize}$. If one or more cells are present in a bin, we increment the count in the channel corresponding to the respective cell type. Formally, for each cell:

- Determine its bin index $(x_{\text{bin}}, y_{\text{bin}})$.
- Increment $\text{image}[c, y_{\text{bin}}, x_{\text{bin}}]$ by $\frac{1}{r_x r_y}$.

By dividing by $g_x g_y$, we normalize the count so that the value in each bin represents an area contribution, ensuring that our image values stay in the range $[0, 1]$. This produces an image tensor of shape $(\text{num cell types}, \text{gridsize}, \text{gridsize})$. Thus, for our three cell types, we can represent the data by images, as shown in figure 2.

For substrate channels, the process is similar but simpler, since substrate values are continuous. Using the same $(x_{\text{bin}}, y_{\text{bin}})$ discretizations, we assign each bin the maximum substrate value observed in that bin. A min-max normalization is then applied across the entire image for each substrate channel, bringing all values into range $[0, 1]$, which can be seen in figure 3.

Multi-Channel Cell Images (img_mc_cells). This observation space extends the spatial discretization described earlier by encoding each cell type into a separate image channel. After discretization, the image tensor is

$$s_t^{\text{mc-cells}} \in \mathbb{N}^{N_c \times g_x \times g_y},$$

where each channel represents the normalized spatial density of one cell type, rescaled to the range $[0, 255]$.

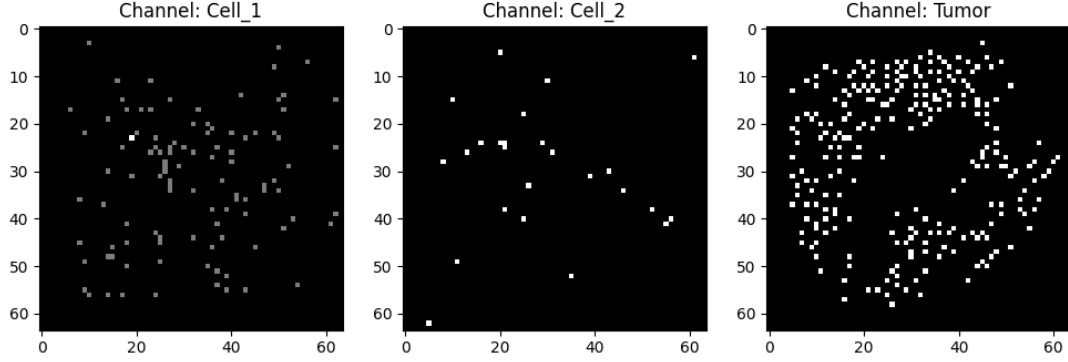


Figure 2: Representation of different channels, each channel represents a cell type.

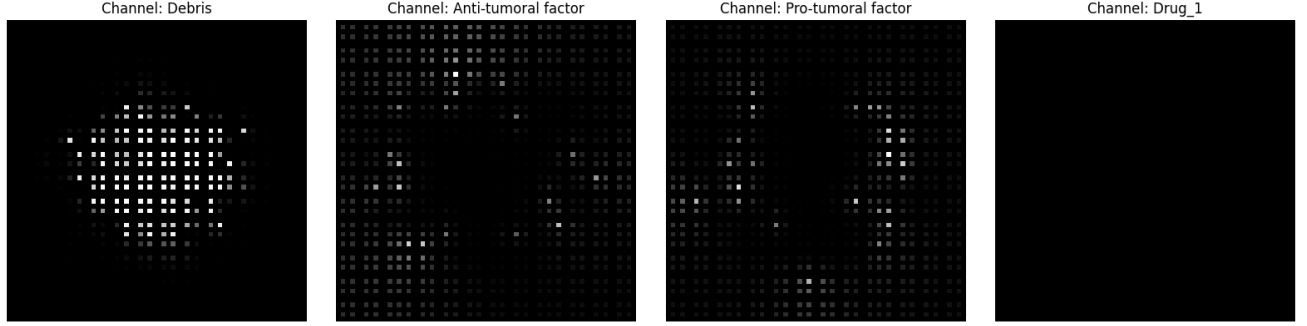


Figure 3: Representation of different channels; each channel represents a substrate.

Multi-Channel Substrate Images (`img_mc_substrates`). Similarly, each substrate is represented as a separate image channel. For each grid cell, the maximum substrate concentration observed in that bin is assigned. A min-max normalization is applied independently to each substrate channel:

$$\tilde{s}_j(x, y) = \frac{s_j(x, y) - \min s_j}{\max s_j - \min s_j},$$

yielding an image tensor

$$s_t^{\text{mc-sub}} \in \mathbb{N}^{N_s \times g_x \times g_y}.$$

Multi-Channel Cells and Substrates (`img_mc_cells_substrates`). This observation space concatenates the cell and substrate image channels along the channel dimension:

$$s_t^{\text{mc-cells+sub}} \in \mathbb{N}^{(N_c + N_s) \times g_x \times g_y}.$$

2.4.3 Graph-Based Observation Spaces

Cell Interaction Graphs (`graph_delaunay` and `graph_knn`). In these observation spaces, the cellular population is represented as a graph

$$\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t),$$

where each node $v \in \mathcal{V}_t$ corresponds to a living cell, and edges are constructed either via Delaunay triangulation or k -nearest neighbors in the (x, y) plane.

Each node is associated with a feature vector

$$\mathbf{x}_v = \frac{\text{type}(v)}{N_c},$$

encoding the normalized cell type. Each edge $(u, v) \in \mathcal{E}_t$ carries a scalar attribute corresponding to the normalized Euclidean distance between the two cells:

$$\mathbf{e}_{uv} = \frac{\|\mathbf{p}_u - \mathbf{p}_v\|}{\max(\text{width}, \text{height}, \text{depth})}.$$

To allow batching, graphs are padded to fixed sizes with $N_{\max}$ nodes and $E_{\max}$ edges. Binary masks indicate valid nodes and edges. The final observation is given by the tuple

$$s_t^{\text{graph}} = (\mathbf{X}, \mathbf{E}, \mathbf{A}, \mathbf{m}_V, \mathbf{m}_E),$$

where $\mathbf{X} \in \mathbb{R}^{N_{\max} \times d_V}$ are node features, $\mathbf{A} \in \mathbb{N}^{2 \times E_{\max}}$ is the edge index tensor, $\mathbf{E} \in \mathbb{R}^{E_{\max} \times d_E}$ are edge attributes, and $\mathbf{m}_V, \mathbf{m}_E$ are node and edge masks, respectively.

2.5 Action A & Reward R

In reinforcement learning, the reward function plays a central role, as it provides the learning signal that links actions to their long-term consequences on the environment. An inadequately designed reward may lead to misleading gradients and suboptimal or unstable policies, even when the state and action spaces are well defined. In our setting, reward design is particularly critical, as the objective involves a delicate trade-off between reducing tumor burden and limiting the amount of drug introduced into the tumor microenvironment.

In the literature, numerous reward or cost formulations have been proposed for chemotherapy scheduling and related clinical control problems. Many works in optimal control and clinical reinforcement learning define the objective as a combination of disease burden and treatment cost. Classical optimal control studies consider objectives of the form.

Many works in optimal chemotherapy scheduling and clinical reinforcement learning formulate the reward or cost function as a combination of disease burden and treatment cost. Classical optimal control studies use objectives of the form $\int_0^T (w_1 C(t) + w_2 u(t)) dt$, with $C(t)$ denoting tumor size and $u(t)$ the administered drug, to balance tumor reduction and toxicity [13]. In reinforcement learning settings, similar formulations appear in the context of adaptive dosing, including $r_t = -C_t - \alpha u_t$ for chemotherapy and analogous expressions in sepsis management [24]. In this work, we propose a novel reward formulation that extends this paradigm by normalizing tumor response relative to the expected untreated growth and by introducing an explicit penalty on dosage escalation, thereby capturing both cumulative exposure and toxicity associated with abrupt increases in drug administration.

At each control step t , the agent selects a drug administration action

$$u_t \in [0, 1],$$

where $u_t = 0$ corresponds to no drug administration and $u_t = 1$ corresponds to the maximum admissible drug concentration applied to the tumor microenvironment. The action space is therefore defined as:

$$A = [0, 1].$$

Let c_t denote the total number of tumor cells at time t . For our basic model and in the absence of treatment, tumor growth is assumed to follow an exponential model [31]:

$$\frac{dc(t)}{dt} = \lambda c(t),$$

We can derivate an expected untreated tumor growth over one control interval Δt :

$$\Delta c_t^{\text{exp}} = c_t (e^{\lambda \Delta t} - 1).$$

We used this term as a normalizing factor Δc_t^{exp} . We can define a normalized tumor growth:

$$r_{\text{tumor}}(t) = \frac{c_t - c_{t+\Delta t}}{\Delta c_t^{\text{exp}}},$$

which measures the reduction in tumor size relative to its expected natural growth. The tumor growth rate λ is computed as

$$\lambda = \text{cycle} \ln(2) - d,$$

where the parameters are specified in `PhysiCell.Settings.xml`. Here, $\text{cycle} \in R^+$ denotes the cell-cycle transition rate (division rate), and $d \in R^+$ denotes the cell death rate.

In addition to tumor suppression, drug usage is penalized by the mean administered dose:

$$r_{\text{amount}}(t) = u_t,$$

and an additional penalty is introduced for dosage escalation between consecutive time steps:

$$r_{\text{increase}}(t) = \max(0, u_t - u_{t-1}).$$

The final reward function is defined as:

$$r_t = w_{\text{tumor}} r_{\text{tumor}}(t) - w_{\text{amount}} r_{\text{amount}}(t) - w_{\text{increase}} r_{\text{increase}}(t),$$

where $w_{\text{tumor}}, w_{\text{amount}}, w_{\text{increase}} \geq 0$ are weighting coefficients satisfying:

$$w_{\text{tumor}} + w_{\text{amount}} + w_{\text{increase}} = 1.$$

The values of weights are: $w_{\text{tumor}} = 0.7$, $w_{\text{increase}} = 0.2$, $w_{\text{amount}} = 0.1$. The weight w_{tumor} controls the tumor eradication, while w_{amount} and w_{increase} regulate overall drug exposure and temporal smoothness of the treatment protocol, respectively. Increasing w_{tumor} prioritizes tumor suppression at the cost of higher drug administration, whereas increasing w_{amount} or w_{increase} encourages more conservative dosing strategies. Importantly, the proposed reward formulation is generic and reusable. It does not introduce environment-specific hyperparameters that require tuning for stability or convergence. Instead, the weights reflect only the user’s therapeutic preference between tumor control and drug toxicity. As a result, this reward remains robust across different temporal resolutions and modeling assumptions, while consistently encouraging both tumor reduction and minimized drug administration.

To address our control objective maximizing cancer cell eradication while minimizing drug administration we employ Soft Actor-Critic (SAC) [9], a state-of-the-art reinforcement learning algorithm for continuous control. SAC is particularly well suited to our setting due to its high sample efficiency, enabling effective learning from a limited number of environment interactions. Moreover, as a deep reinforcement learning method, SAC accommodates different observation types, such as image-based representations, through the use of Convolutional Neural Networks (CNN) [20] or graphs by the use of Graph Neural Networks (GNN)[27].

A key challenge in our work is the computational cost of the simulator. To alleviate this bottleneck, we adopt an asynchronous parallel training framework inspired by Mnih et al. [18], and specifically leverage the Asynchronous SAC formulation introduced in Liu et al. [15].

In this framework, multiple environments are executed concurrently using a decoupled architecture. A dedicated learning process performs policy and value updates on the GPU, while multiple environment processes run in parallel on the CPU to generate experience. These processes operate continuously and asynchronously, without global synchronization, enabling efficient utilization of computational resources.

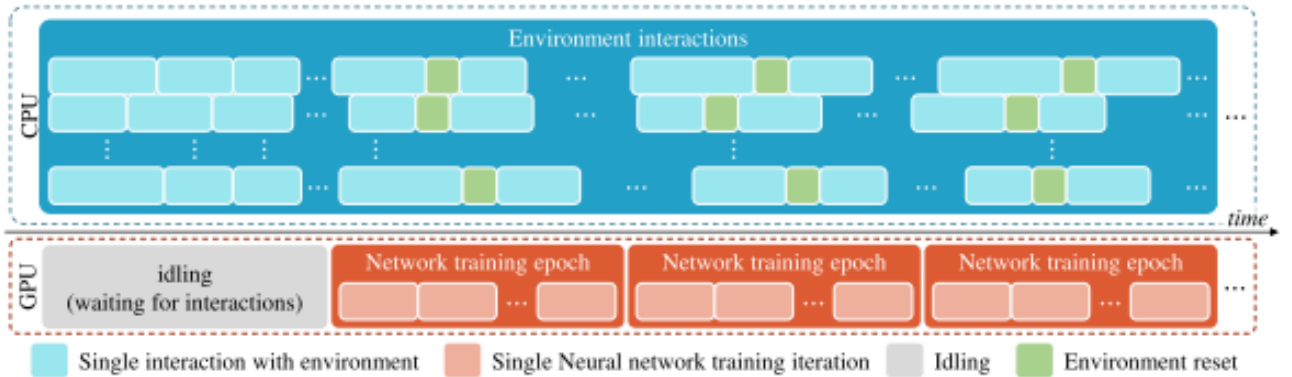


Figure 4: Asynchronous parallel training figure from [15]

3 Results

In our experimental evaluation, we compare multiple state-space representations with the hyperparameters of the Asynchronous SAC algorithm fixed as described in Table 6. As demonstrated in Figure 5, we observe at the beginning learning possible faster for scalars state space but around 0.75×10^5 training steps we observe than

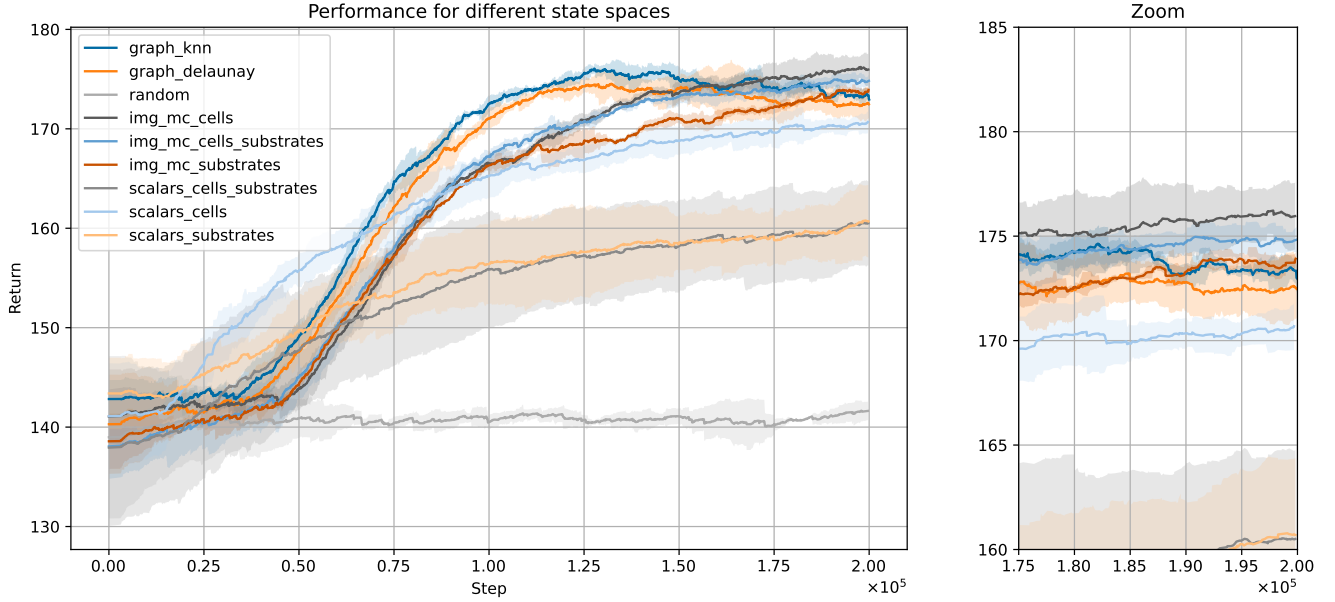


Figure 5: Performance for different state spaces X-axis represents the number of steps while Y-axis represents the return across 5 different seeds.

Table 1: Final performance metrics for different observation modalities. Values are reported at the final evaluation step as mean (μ), standard deviation (σ) and relative variation noted % (computed according to: $\frac{\mu_{\text{observation_type}} - \mu_{\text{random}}}{\mu_{\text{random}}}$)

Observation type	μ	σ	%
Random	141.63	0.94	0.0
Scalars (cells)	170.69	1.14	20.52
Scalars (substrates)	160.70	3.68	13.46
Scalars (cells + substrates)	160.50	4.26	13.32
Image (Multi-Channel cells)	175.96	1.60	24.24
Image (Multi-Channel substrates)	173.92	0.26	22.80
Image (Multi-Channel cells + substrates)	174.83	0.61	23.44
Graph (kNN with k=3)	172.95	0.73	22.11
Graph (Delaunay)	172.47	1.40	21.78

graph and image state spaces are superior to scalars state space in terms of return and around 1.00×10^5 . In terms of sample efficiency, state spaces related to space are more sample efficient than scalars state space.

As shown in Table 1, in mean cumulative return across three seeds, image and graph based state spaces which encode spatial information are overall superior to scalar-based observations after 2.00×10^5 training steps. In particular, image-based representations achieve the highest performance, in particularly multi-channel cells attaining the best mean cumulative return. Graph remains better than scalar observations but they are outperformed by image observations in long run. However between 0.5×10^5 and 1.375×10^5 have a better mean cumulative return than any observation type. For both image and scalar based observations, using solely cell information provides the best performance compared to including substrate information or their combination. In particular, for the scalars the impact is significative, while for image it is less pronounced.

In terms of stability, space-based observations consistently exhibit lower standard deviation across seeds, indicating more robust learning dynamics.

4 Discussion

This study demonstrates that PhysiGym enables reinforcement learning agents to effectively control an agent-based tumor microenvironment model through a standardized interface. Beyond validating the framework itself, our experiments highlight the critical role of state representation in learning dynamic treatment strategies.

Across all experiments, observation spaces that preserve spatial information outperform scalar representations. This finding aligns with the intuition that biological systems are inherently spatial, and that aggregating

the environment into global statistics removes information essential for decision-making. By retaining spatial structure, image- and graph-based representations allow the agent to reason about local interactions, spatial heterogeneity, and emergent patterns that influence tumor progression and treatment response.

Interestingly, within both scalar and image-based categories, representations based solely on cell information outperform those that include substrate concentrations. In the context of our simplified model, this suggests that cell populations provide a sufficient summary of the system state for control, while substrate information may be redundant or insufficiently informative when encoded in a compressed form. However, this result should not be interpreted as a general statement about biological systems. In more complex or realistic models, substrates may play a more decisive role, particularly if they are measured or encoded with higher fidelity.

From a practical perspective, the strong performance of cell-only representations is encouraging. In experimental or clinical settings, direct measurement of molecular concentrations is often difficult, noisy, or expensive, whereas cell counts and spatial organization may be more accessible. Our results suggest that effective control strategies may be learned even under limited observability, provided that key spatial features are retained.

Despite these promising findings, several limitations must be acknowledged. First, the tumor microenvironment model used in this work is intentionally simplified and serves primarily as a controlled testbed. Real biological systems exhibit greater heterogeneity, stochasticity, and multiscale dynamics, and extending PhysiGym to more realistic three-dimensional models will be necessary to assess the generality of the observed trends. Second, all experiments rely on a single reinforcement learning algorithm, Soft Actor-Critic. While SAC is well suited for continuous control, alternative algorithms or model-based approaches may interact differently with the various state representations.

More broadly, PhysiGym opens several research directions at the intersection of agent-based modeling and reinforcement learning. From a biological standpoint, it provides a flexible platform for exploring adaptive treatment strategies *in silico* before moving to experimental validation. From a machine learning perspective, it enables systematic benchmarking of algorithms and representations on structured, spatially explicit environments. By lowering the barrier between biological simulation and modern reinforcement learning frameworks, PhysiGym facilitates reproducible and extensible research across disciplines.

In summary, our results show that preserving spatial information is key to effective reinforcement learning control in agent-based biological simulations. PhysiGym provides the necessary infrastructure to study this question in a principled and reproducible manner, and we anticipate that it will serve as a valuable tool for both the computational biology and reinforcement learning communities.

5 Conclusion

PhysiGym provides a novel framework that bridges reinforcement learning with agent-based biological simulations, enabling the study of dynamic treatment regimes *in-silico*. Our results indicate that leveraging spatial representations improves policy performance, underscoring the importance of biologically meaningful state spaces. By making PhysiGym open-source and extensible, we aim to foster collaboration between computational scientists and biologists, paving the way toward AI-driven strategies for personalized cancer therapy. We anticipate that PhysiGym will serve as a valuable testbed for both communities to advance decision-support systems that could translate into clinical practice in the future.

6 Code Management and Availability

PhysiGym is an open-source project, and contributions from the community are encouraged. New PhysiCell models can be integrated by defining the required cell types and interactions, ensuring compatibility with PhysiGym’s step-based execution, exposing action and observation spaces in Python, and wrapping the model with a Gymnasium-compatible environment. Source code, examples, and documentation are available at <https://github.com/Dante-Berth/PhysiGym>.

Installation and troubleshooting how-tos, a modeling tutorial, a RL tutorial, and a reference manual can be found at [<https://github.com/Dante-Berth/PhysiGym/tree/main/man>].

7 Competing interests

No competing interest is declared.

8 Acknowledgments

A. B., V.P., and E. R. acknowledge funding from Region Occitanie and INSERM, Projet Emergence REACT. E. B. received support from a Chateaubriand Fellowship of the Office for Science & Technology of the Embassy of

France in the United States. This study has been partially supported through the grant EUR CArE N°ANR-18-EURE-0003 in the framework of the Programme des Investissements d'Avenir and an Eiffel Excellence doctoral fellowship to M. H. This research was also supported in part by Lilly Endowment, Inc., through its support for the Indiana University Pervasive Technology Institute.

References

- [1] S. Aif, M. Eiche, N. Appold, E. Fischer, T. Citak, and J. Kayser. Reinforcement failing guides the discovery of emergent spatial dynamics in adaptive tumor therapy. Apr. 2025. doi: 10.1101/2025.04.08.647768.
- [2] A. G. Barto, R. S. Sutton, and C. Watkins. *Learning and sequential decision making*, volume 89. University of Massachusetts Amherst, MA, 1989.
- [3] R. Bellman. A markovian decision process. *Journal of mathematics and mechanics*, pages 679–684, 1957.
- [4] B. Chakraborty and S. A. Murphy. Dynamic treatment regimes. *Annual review of statistics and its application*, 1(1):447–464, 2014.
- [5] L. Espeholt, H. Soyer, R. Munos, K. Simonyan, V. Mnih, T. Ward, Y. Doron, V. Firoiu, T. Harley, I. Dunning, et al. Impala: Scalable distributed deep-rl with importance weighted actor-learner architectures. In *International conference on machine learning*, pages 1407–1416. PMLR, 2018.
- [6] A. Ghaffarizadeh, S. H. Friedman, and P. Macklin. Biofvm: an efficient, parallelized diffusive transport solver for 3-d biological simulations. *Bioinformatics*, 32(8):1256–1258, 2016.
- [7] A. Ghaffarizadeh, R. Heiland, S. H. Friedman, S. M. Mumenthaler, and P. Macklin. PhysiCell: An openb source physics-based cell simulator for 3-d multicellular systems. *PLOS Computational Biology*, 14(2): e1005991, feb 2018. doi: 10.1371/journal.pcbi.1005991.
- [8] J. W. Gray and M. G. Pallavicini. Ara-c scheduling: Theoretical and experimental considerations. *Medical and Pediatric Oncology*, 10(S1):93–108, 1982. ISSN 1096-911X. doi: 10.1002/mpo.2950100711.
- [9] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, et al. Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*, 2018.
- [10] R. Heiland, D. Bergman, B. Lyons, G. Waldow, J. Cass, H. L. da Rocha, M. Ruscone, V. Noël, and P. Macklin. Physicell studio: a graphical tool to make agent-based modeling more accessible. *Gigabyte*, 2024:gigabyte128, 2024.
- [11] J. A. Johnson, D. R. Bergman, H. L. Rocha, D. L. Zhou, E. Cramer, I. C. Mclean, Y. W. Dance, M. Booth, Z. Nicholas, T. Lopez-Vidal, A. Deshpande, R. Heiland, E. Bucher, F. Shojaeian, M. Dunworth, A. Forjaz, M. Getz, I. Godet, F. Kurtoglu, M. Lyman, J. Metzcar, J. T. Mitchell, A. Raddatz, J. Solorzano, A. Sundus, Y. Wang, D. G. DeNardo, A. J. Ewald, D. M. Gilkes, L. T. Kagohara, A. L. Kiemen, E. D. Thompson, D. Wirtz, L. D. Wood, P.-H. Wu, N. Zaidi, L. Zheng, J. W. Zimmerman, J. M. Phillip, E. M. Jaffee, J. W. Gray, L. M. Coussens, Y. H. Chang, L. M. Heiser, G. L. Stein-O’Brien, E. J. Fertig, and P. Macklin. Human interpretable grammar encodes multicellular systems biology models to democratize virtual cell laboratories. *Cell*, July 2025. ISSN 0092-8674. doi: 10.1016/j.cell.2025.06.048.
- [12] R. Laubenbacher, F. Adler, G. An, F. Castiglione, S. Eubank, L. L. Fonseca, J. Glazier, T. Helikar, M. Jett-Tilton, D. Kirschner, et al. Toward mechanistic medical digital twins: some use cases in immunology. *Frontiers in Digital Health*, 6:1349595, 2024.
- [13] U. Ledzewicz, S. Wang, H. Schättler, N. André, M. A. Heng, and E. Pasquier. On drug resistance and metronomic chemotherapy: A mathematical modeling and optimal control approach. *Mathematical Biosciences & Engineering*, 14(1):217–235, 2016.
- [14] G. Letort, A. Montagud, G. Stoll, R. Heiland, E. Barillot, P. Macklin, A. Zinovyev, and L. Calzone. Physi-BoSS: a multi-scale agent-based modelling framework integrating physical dimension and cell signalling. *Bioinformatics*, 35(7):1188–1196, aug 2018. doi: 10.1093/bioinformatics/bty766.
- [15] X.-Y. Liu, D. Bodaghi, Q. Xue, X. Zheng, and J.-X. Wang. Asynchronous parallel reinforcement learning for optimizing propulsive performance in fin ray control. *Engineering with Computers*, 41(4):2201–2218, 2025.
- [16] J. Metzcar, Y. Wang, R. Heiland, and P. Macklin. A review of cell-based computational modeling in cancer biology. *JCO Clinical Cancer Informatics*, pages 1–13, Dec. 2019. ISSN 2473-4276. doi: 10.1200/cci.18.00069.
- [17] D. Misra. Mish: A self regularized non-monotonic activation function. *arXiv preprint arXiv:1908.08681*, 2019.

- [18] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International conference on machine learning*, pages 1928–1937. PmLR, 2016.
- [19] V. Noël, M. Ruscone, R. Shuttleworth, and C. K. Macnamara. Physimess - a new physicell addon for extracellular matrix modelling. *Gigabyte*, 2024, Oct. 2024. ISSN 2709-4715. doi: 10.46471/gigabyte.136.
- [20] K. O’shea and R. Nash. An introduction to convolutional neural networks. *arXiv preprint arXiv:1511.08458*, 2015.
- [21] M. Ponce-de Leon, A. Montagud, V. Noël, A. Meert, G. Pradas, E. Barillot, L. Calzone, and A. Valencia. Physiboss 2.0: a sustainable integration of stochastic boolean and agent-based modelling frameworks. *npj Systems Biology and Applications*, 9(1):54, 2023.
- [22] M. L. Puterman. *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, 2014.
- [23] Python Software Foundation. Extending and embedding the python interpreter. <https://docs.python.org/3/extending/embedding.html>, 2024. Accessed: 2024-02-07.
- [24] A. Raghu, M. Komorowski, I. Ahmed, L. Celi, P. Szolovits, and M. Ghassemi. Deep reinforcement learning for sepsis treatment. *arXiv preprint arXiv:1711.09602*, 2017.
- [25] L. H. Rocha, B. Aguilar, M. Getz, I. Shmulevich, and P. Macklin. A multiscale model of immune surveillance in micrometastases gives insights on cancer patient digital twins. *npj Systems Biology and Applications*, 10(1), Dec. 2024. ISSN 2056-7189. doi: 10.1038/s41540-024-00472-z.
- [26] M. Ruscone, A. Checcoli, R. Heiland, E. Barillot, P. Macklin, L. Calzone, and V. Noël. Building multiscale models with physiboss, an agent-based modeling tool. *Briefings in Bioinformatics*, 25(6), Sept. 2024. ISSN 1477-4054. doi: 10.1093/bib/bbae509.
- [27] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini. The graph neural network model. *IEEE transactions on neural networks*, 20(1):61–80, 2008.
- [28] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG, R. Perez-Vicente, A. Pierré, S. Schulhoff, J. J. Tai, H. Tan, and O. G. Younis. Gymnasium: A standard interface for reinforcement learning environments, 2024.
- [29] G. van Rossum. Extending and embedding the python interpreter. *CWI Report CS-R9527*, May 1995. URL <https://gvanrossum.github.io/Publications.html>.
- [30] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, Y. Bengio, et al. Graph attention networks. *stat*, 1050(20):10–48550, 2017.
- [31] Y. Watanabe, E. L. Dahlman, K. Z. Leder, and S. K. Hui. A mathematical model of tumor growth and its response to single irradiation. *Theoretical Biology and Medical Modelling*, 13(1):6, 2016.
- [32] C. J. Watkins and P. Dayan. Q-learning. *Machine learning*, 8:279–292, 1992.
- [33] A. E. Zade, S. S. Haghighi, and M. Soltani. Reinforcement learning for optimal scheduling of glioblastoma treatment with temozolomide. *Computer methods and programs in biomedicine*, 193:105443, 2020.

A Generation of Spatially Correlated Initial Conditions

This appendix describes the procedure used to generate spatially structured initial conditions for synthetic cellular networks. The method constructs spatially correlated random fields that bias the placement of different cell types while preserving stochasticity at both global and local scales.

A.1 Gaussian-Correlated Random Fields

Let $\Omega \subset \mathbb{Z}^2$ denote a two-dimensional spatial domain of size $N_x \times N_y$. We first generate a spatially correlated scalar field by filtering white Gaussian noise with a Gaussian kernel.

A white noise field $\eta(\mathbf{x})$ is sampled such that

$$\eta(\mathbf{x}) \sim \mathcal{N}(0, 1), \quad \mathbf{x} \in \Omega.$$

This noise is then convolved with a Gaussian kernel G_σ ,

$$G_\sigma(\mathbf{r}) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{\|\mathbf{r}\|^2}{2\sigma^2}\right),$$

yielding the correlated field

$$F(\mathbf{x}) = (G_\sigma * \eta)(\mathbf{x}).$$

The standard deviation of the kernel is chosen as $\sigma = \frac{\ell_c}{\sqrt{2}}$, where ℓ_c is the prescribed spatial correlation length. This choice ensures that the resulting field exhibits correlations that decay approximately over length scale ℓ_c .

Reflective boundary conditions are applied during filtering to avoid edge artifacts.

A.2 Balanced Fields with Local Fluctuations

For each cell type i , a distinct spatial field $F_i(\mathbf{x})$ is constructed by combining a global correlated field with an additional locally filtered noise component.

A local noise field $\xi_i(\mathbf{x})$ is drawn independently:

$$\xi_k(\mathbf{x}) \sim \mathcal{N}(0, 1),$$

and filtered with a narrower Gaussian kernel of width $\ell_c/3$:

$$\tilde{\xi}_k(\mathbf{x}) = (G_{\ell_c/3} * \xi_k)(\mathbf{x}).$$

The final field for cell type i is defined as

$$F_i(\mathbf{x}) = A \left[F(\mathbf{x}) + \alpha \tilde{\xi}_k(\mathbf{x}) \right],$$

where A is a global amplitude parameter and α controls the relative strength of local fluctuations. In practice, α is set as a small fraction of the global field amplitude to preserve large-scale structure while introducing local heterogeneity.

Each field is then normalized and inverted to produce a bounded spatial preference map:

$$\hat{F}_i(\mathbf{x}) = \frac{\max(F_i) - F_i(\mathbf{x})}{\max(F_i) - \min(F_i)}.$$

This normalization ensures $\hat{F}_i(\mathbf{x}) \in [0, 1]$, with larger values corresponding to higher spatial affinity.

A.3 Probabilistic Cell Placement

Given a normalized field $\hat{F}_i(\mathbf{x})$, candidate spatial locations are restricted to pixels satisfying

$$\hat{F}_i(\mathbf{x}) > \theta_i,$$

where θ_i is a cell-type-specific threshold.

From these locations, N_i cell positions are sampled without replacement using a probability distribution proportional to the field intensity:

$$P_k(\mathbf{x}) = \frac{\hat{F}_i(\mathbf{x})}{\sum_{\mathbf{y} \in \Omega_i} \hat{F}_i(\mathbf{y})}, \quad \mathbf{x} \in \Omega_i,$$

where $\Omega_i = \{\mathbf{x} \in \Omega \mid \hat{F}_i(\mathbf{x}) > \theta_i\}$.

This weighted sampling biases cell placement toward regions of high field intensity while maintaining stochastic variability.

A.4 Final Synthetic Network

The procedure is repeated independently for each cell type, producing a multi-phenotype spatial configuration. The final output is a set of three-dimensional cell coordinates (x_i, y_i, z_i) with $z_i = 0$ and associated categorical labels indicating cell phenotype.

This approach yields spatially heterogeneous yet statistically controlled initial conditions suitable for simulating tissue organization, cell-cell interactions, or spatial signaling networks.

A.5 Hyperparameters for synthetic network generation

Table 2: Global parameters for synthetic network generation.

Category	Parameter	Symbol / Name	Value / Description
Cell type: Tumor	Correlation length	ℓ_c^{tumor}	45
	Threshold	θ_{tumor}	0.55
	Number of cells	N_{tumor}	512
Cell type: Cell 1	Correlation length	$\ell_c^{\text{cell}_1}$	45
	Threshold	θ_{cell_1}	0.55
	Number of cells	N_{cell_1}	128
Global	Cell 2 fraction	f_{cell_2}	{0.0, 0.25, 0.5, 0.75, 1.0}
Global	Random seed	$seed$	Inherited from simulation seed

The Cell 2 fraction controls the proportion of cell₂ randomly replaced by cell₁, or alternatively, cell₂ can be generated using the same spatial parameters as tumor and cell₁.

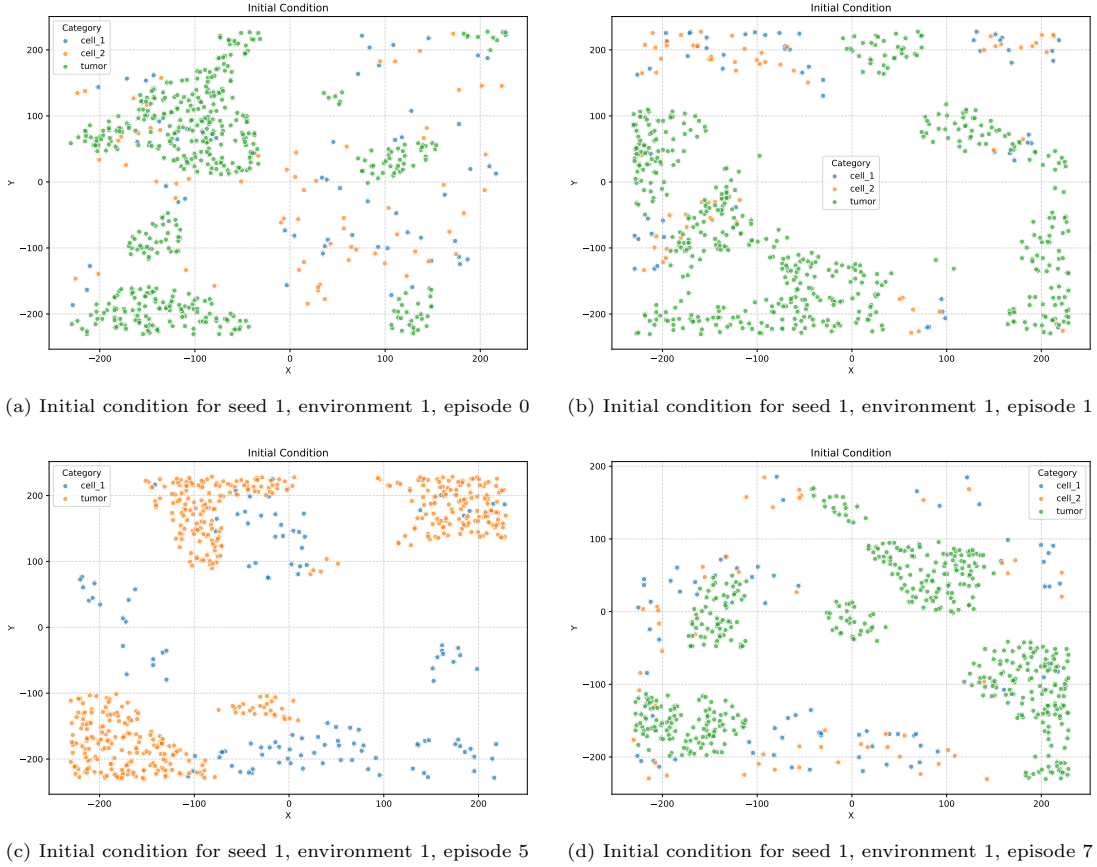


Figure 6: Four generated initial conditions.

B Neural architecture description

B.1 Neural Network Architecture

We employ a modular neural architecture following an actor-critic paradigm, designed to accommodate heterogeneous observation modalities, including image-based, vector-based, and graph-structured state representations. A shared feature extraction module encodes observations into a common latent representation, which is then used by separate policy (actor) and value (critic) networks.

The feature extractor dynamically adapts to the observation type. Image-based inputs are processed through convolutional encoders, graph-structured inputs are handled by attention-based graph neural networks, and vector observations are passed directly without transformation. In all cases, the extracted features are flattened into a fixed-dimensional latent vector that serves as the input to both actor and critic networks.

B.1.1 Convolutional Feature Extractors

We have considered to use an IMPALA-style Convolutional Network for image-based inputs [5].

IMPALA-style Convolutional Network. The first architecture is inspired by the IMPALA CNN design and consists of a stack of convolutional blocks. Each block comprises:

- A 3×3 convolution,
- A 3×3 max-pooling operation with stride 2,
- Two residual blocks with skip connections.

Each residual block applies two convolutional layers with Mish [17] nonlinearities and preserves the input through identity shortcuts. This structure promotes stable gradient flow and enables deep feature hierarchies.

B.1.2 Graph-Based Feature Extraction

For graph-structured observations, we employ a graph attention network (GAT) [30]. The extractor consists of two stacked GAT layers with multi-head attention in the first layer, followed by a single-head aggregation layer. Node embeddings are aggregated using global mean pooling to produce a fixed-dimensional graph-level representation. As activation function, we use Mish.

B.1.3 Actor-Critic Networks

Both the actor and critic networks share a common architectural backbone following feature extraction. In each case, the extracted state representation is processed by a multilayer perceptron consisting of two fully connected layers of width 256. Nonlinear activations are applied between layers (Mish for the critic and ReLU for the actor) .

C Asynchronous SAC with Parallel Environments

C.1 Computational time

We used a desktop machine (32 threads, 13th Gen Intel(R) Core(TM) i9-13900K) with an NVIDIA GeForce RTX 4090. The number of environments noted N_{env} was 9, and 3 threads were used for a PhysiCell simulation and we allocated the 5 left threads to the reinforcement learning algorithm. Empirically, we observed improved performance when using at least five threads for asynchronous SAC. Table 3 reports the time required for different state space representations. Graph-based state spaces are the slowest, requiring more than two hours, whereas scalar state spaces are the fastest, taking less than one hour. Image-based state spaces exhibit intermediate performance, with a runtime of approximately one hour and thirty minutes.

Table 3: Mean Timestamp per State Space

State Space	Mean Timestamp (s)
Random	2893.75
Scalars (cells)	3072.50
Scalars (substrates)	3113.17
Scalars (cells + substrates)	3133.45
Image (MC cells)	5160.60
Image (MC substrates)	5175.52
Image (MC cells + substrates)	5604.56
Graph (kNN with k=3)	8202.35
Graph (Delaunay)	8525.22

Table 4 illustrates the impact of varying the number of environments coupled with an appropriate choice of threads. The optimal number of threads depends on the complexity of the model: for more complex models, fewer environments per thread may be preferable to maximize efficiency. In our setup, using 9 environments with 3 threads per environment allowed us to collect 25000 steps in approximately 6 minutes. In contrast, running a single environment without vectorization would require more than 17 minutes. Thanks to vectorization, the time required to collect samples was effectively reduced by a factor of three, and this reduction could be further improved by increasing the total number of threads in order to increase the number of environments.

Table 5 demonstrates the impact of varying the number of tumor cells on the computational time required

Table 4: The overall number of threads is 27 Mean timestamp per number of environments for collecting 25000 steps

N_{env}	$\mu(s)$	$\sigma(s)$	Threads per environment
1	1059	11	27
2	658	6	13
3	529	3	9
6	438	3	4
9	363	5	3

to collect 25,000 steps in 9 environments using 3 threads. It is clear that increasing the number of tumor cells significantly increases the computation time, with the largest configuration being substantially slower.

Table 5: Computational time for collecting 25,000 steps for 9 environments with 3 threads as a function of the number of tumor cells.

Number of tumor cells (N_{tumor})	Approximate time (min)
512	≈ 6
1024	≈ 10
2048	≈ 30
4096	≈ 180

C.2 Hyperparameters

Table 6: Reinforcement learning algorithm hyperparameters.

Parameter	Symbol / Name	Value / Description
Total training steps	T	2×10^5
Replay buffer size	$ \mathcal{B} $	2×10^5
Batch Size Multiplier	$ \mathcal{BM} $	64
Batch size	B	$ \mathcal{BM} \times N_{\text{env}}$
Warm-up steps	$T_{\text{start}}/\text{learning_starts}$	1e4
Policy update frequency	f_{π}	2
Target network update frequency	f_{target}	1
Entropy autotuning	–	Enabled (Boolean)
Entropy coefficient	α	0.05
Target smoothing coefficient	τ	0.005
Critic learning rate	η_Q	3×10^{-4}
Policy learning rate	η_{π}	3×10^{-4}
Discount factor	γ	0.99
Number of update loops per step	N_{loops}	3

C.3 Algorithm details

Algorithm 1 Asynchronous SAC with Parallel Environments

Require: Simulator configuration d_{arg} , replay buffer size $|\mathcal{B}|$, learning rates, discount factor γ , target update rate τ

Ensure: Trained SAC policy π_θ

```

1: Initialize shared queues: actor_queue, sample_queue
2: Initialize stop signal stop_event
3: Launch Actor Process (CPU)
   Initialize vectorized environments  $\mathcal{E}_1, \dots, \mathcal{E}_N$ 
   Reset environments and obtain initial observations
   Initialize local policy  $\pi_{\theta_a}$ 
4: while not stop_event do
5:   Receive updated policy parameters from actor_queue (if available)
6:   if total steps  $< t_{\text{start}}$  then
7:     Sample random actions
8:   else
9:     Sample actions from  $\pi_{\theta_a}$ 
10:  end if
11:  Step all environments in parallel
12:  Collect transitions  $(s, a, r, s', d)$ 
13:  Send transition batches to sample_queue
14: end while
15: Initialize Learner Process (GPU)
16: Initialize actor  $\pi_\theta$ , critics  $Q_{\phi_1}, Q_{\phi_2}$ 
17: Initialize target critics  $Q_{\phi'_1} \leftarrow Q_{\phi_1}, Q_{\phi'_2} \leftarrow Q_{\phi_2}$ 
18: Initialize replay buffer  $\mathcal{D}$ 
19: Send initial policy parameters  $\theta$  to actor via actor_queue
20: while total samples  $< T$  do
21:   Drain transitions from sample_queue into  $\mathcal{D}$ 
22:   if  $|\mathcal{D}| < \max(t_{\text{start}}, \text{batch size})$  then
23:     Continue
24:   end if
25:   for  $N_{\text{loops}}$  do
26:     for each gradient update step do
27:       Sample minibatch from  $\mathcal{D}$ 
28:       Compute SAC target:
          
$$y = r + \gamma(1 - d) [\min(Q_{\phi'_1}, Q_{\phi'_2}) - \alpha \log \pi_\theta(a'|s')]$$

```

$$y = r + \gamma(1 - d) [\min(Q_{\phi'_1}, Q_{\phi'_2}) - \alpha \log \pi_\theta(a'|s')]$$

```

29:       Update critics  $Q_{\phi_1}, Q_{\phi_2}$ 
30:       Update actor  $\pi_\theta$ 
31:       Update entropy coefficient  $\alpha$  (if enabled)
32:       Soft-update targets:
          
$$\phi' \leftarrow \tau \phi + (1 - \tau) \phi'$$

```

$$\phi' \leftarrow \tau \phi + (1 - \tau) \phi'$$

```

33:     end for
34:   end for
35:   if policy update interval reached then
36:     Send updated policy parameters to actor
37:   end if
38: end while

```

C.4 Bootstrapped Results

The results are derived from the same set of raw learning curves obtained from five different random seeds. Given these five raw curves, we generate 256 bootstrap replicates by resampling the seed dimension with replacement. For each bootstrap replicate, a mean learning curve is computed across the resampled seeds and subsequently smoothed using an exponentially weighted moving average (EWMA) with $\alpha = 0.01$. The reported mean corresponds to the EWMA-smoothed average across the original seeds, while confidence intervals are estimated from the empirical distribution of the bootstrapped EWMA curves.

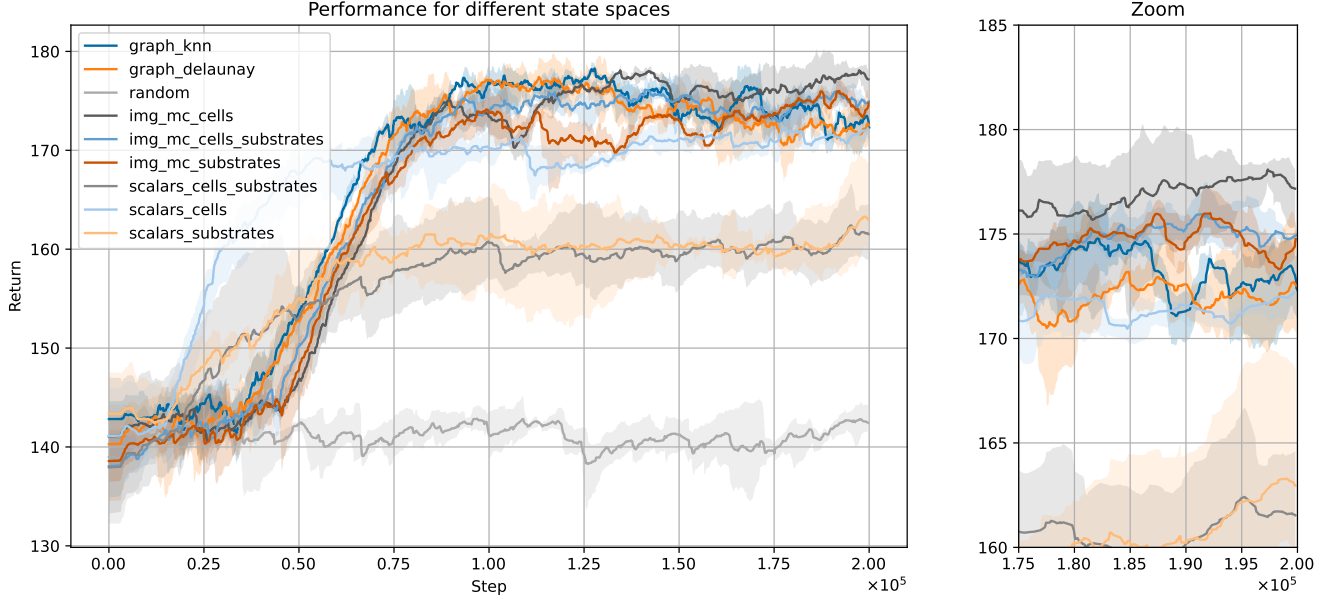


Figure 7: Performance bootstrapped for different state spaces X-axis represents the number of steps while Y-axis represents the return across 5 different seeds.

Observation type	μ	CI low (2.5%)	CI high (97.5%)
Random	142.46	141.42	144.27
Scalars (cells)	172.2	170.54	174.28
Scalars (substrates)	162.96	159.68	168.55
Scalars (cells + substrates)	160.50	156.56	164.6
Image (Multi-Channel cells)	177.17	176.25	178.09
Image (Multi-Channel substrates)	174.78	174.18	175.63
Image (Multi-Channel cells + substrates)	174.95	174.77	175.21
Graph (kNN with k=3)	172.3	170.82	174.07
Graph (Delaunay)	172.52	170.48	175.31

Table 7: Mean performance (μ) with confidence interval bounds for different observation types.

We quantitatively observe the same trends in the corresponding results, consistent with the previous Figure 5.